Day 13/21: Python Inheritance & Polymorphism

The Data Refinery Sprint | Author: Prasad S Hiremath | Topic: Building Reusable Data Pipeline Connectors

⚡ **Daily Sprint Motivation**
Look how far you've come! Day 13 marks passing the halfway checkpoint of our 21-day sprint. Consistency is your ultimate superpower. Today, we master **Inheritance and Polymorphism**—the foundation of extensible framework design in PySpark, Airflow Hooks, and Custom Extractors!

🏢 Real-World Case Study: Duplicate Connectors at InstaDrop

At **InstaDrop**, engineers built separate ingestion pipelines for S3 Buckets, Postgres DBs, and local CSV files. Each class duplicated logging, SSL handshakes, and error retry logic. When authentication requirements changed, engineers had to edit dozens of files. By refactoring to a **BaseIngestion** parent class, they enforced the **DRY (Don't Repeat Yourself)** principle and cut technical debt by 60%.

💡 Core Concepts Explained Simply

1. What is Inheritance? (Parent & Child Blueprint)

Inheritance allows a **Child class** to automatically receive attributes and methods from a **Parent class**. You write shared logic once in the parent, while child classes add specialized features.

2. Method Overriding & Polymorphism

A child class can override a parent method to customize its behavior. **Polymorphism** allows pipelines to process different child classes (e.g., S3, CSV, Postgres) through a single unified interface call like `connector.extract()`.

3. Supercharging with `super()`

The `super()` function allows a child class method to execute the parent's logic before adding its own custom processing steps, preserving foundational setup.

📊 Architecture Comparison in Data Engineering

Evaluation Metric	Monolithic Duplicate Classes	Hierarchical OOP Architecture
Code Reusability	Zero reuse; connection & retry logic copy-pasted across scripts.	High reuse; parent base class encapsulates shared operations.
Maintenance Cost	High risk; bugs must be fixed manually in every connector.	Low cost; fixes in Parent class update all child pipelines.
Polymorphic Ingestion	Requires custom conditional branching for each data source.	Unified `extract()` call handles any inherited connector.

💻 Medium-Level Coding Challenge: InstaDrop Ingestion Engine

> **Task:** Build a `BaseIngestion` parent class that manages source names and logging, and derive a `CSVIngestion` child class that overrides `extract_data()` using `super()`.

```
class BaseIngestion: """Base parent class for all InstaDrop data connectors.""" def __ini
```

❓ Part 1: Core Concept MCQs

Q1: What is the primary operational benefit of Inheritance in Python pipelines?

A) Doubles CPU execution speed | B) Eliminates code duplication by sharing common parent logic
C) Automatically indexes SQL database columns | D) Bypasses network security firewalls

Key: B — Reasoning: Inheritance enforces the DRY principle by allowing child classes to inherit shared attributes and methods.

Q2: What is the correct syntax for declaring class `PostgresHook` inheriting from `BaseHook`?

A) class PostgresHook extends BaseHook: | B) class PostgresHook(BaseHook):
C) class PostgresHook inherits BaseHook: | D) class PostgresHook -> BaseHook:

Key: B — Reasoning: Python uses parent class names inside parentheses directly following the child class name declaration.

❓ Core Concept MCQs (Continued)

Q3: How does Method Overriding work in a child class?

A) Deletes the parent class permanently | B) Redefines a method with the same name as the parent to alter behavior
C) Prevents the child from instantiating | D) Converts method outputs into JSON objects

Key: B — Reasoning: Overriding allows a child class to replace or customize specific functionality inherited from a parent.

Q4: What is the purpose of invoking `super().__init__()` inside a child constructor?

A) Restarts Python runtime | B) Deletes child properties
C) Calls the parent class constructor to initialize base attributes | D) Casts all strings to numeric values

Key: C — Reasoning: 'super()' delegates construction tasks to the parent class, ensuring base state is correctly configured.

Q5: In multi-level inheritance (`Class C -> Class B -> Class A`), what can Class C access?

A) Attributes in Class C only | B) Attributes in Class C and B only
C) Public attributes/methods from Class C, Class B, and Class A | D) None (Inheritance chain breaks)

Key: C — Reasoning: Inheritance cascades downwards; Class C inherits all accessible properties up the entire ancestor chain.

💡 Part 2: 5 Bonus Q&A (Advanced Pipeline Focus)

Bonus Q1: What is Method Resolution Order (MRO) and C3 Linearization?

Answer: MRO determines the sequence Python follows to search for methods/attributes in multiple inheritance hierarchy. Python uses the **C3 Linearization algorithm**, viewable via `Class.mro()` or `Class.__mro__`, ensuring parent classes are inspected after child classes without cycle duplication.

Bonus Q2: How do Abstract Base Classes (ABC) enforce interface contracts in pipelines?

Answer: By inheriting from `abc.ABC` and decorating methods with `@abstractmethod`, Python prevents instantiation of the base class unless the child overrides all decorated methods. Frameworks like Airflow use this to enforce that every operator implements an `execute()` method.

Bonus Q3: What is Duck Typing and how does it enable Polymorphism in Python?

Answer: Duck typing ("If it walks like a duck...") means Python evaluates objects by their methods rather than explicitly inherited types. A pipeline runner can iterate through a list of objects and execute `obj.extract()` without checking explicit subclassing.

Bonus Q4: Why is `isinstance(obj, BaseClass)` preferred over `type(obj) == BaseClass`?

Answer: `isinstance()` supports inheritance by evaluating to `True` for instances of derived child classes, whereas direct `type()` comparison fails on subclassed instances.

Bonus Q5: How are private attributes (`__attr`) inherited by child classes?